

Manual de Uso

Claude Code • Hermes Stack

Tu referencia rápida. Diseñado para escanear, no para leer.

Actualizado: 25 de mayo de 2026

Panel de Control — Respuesta en <5 segundos

Cuando necesites algo, aquí está la acción correcta:

Situación	Comando / Acción	Por qué
Inicio de sesión	<code>make doctor</code>	Estado completo del stack en 1 comando
¿Qué tengo pendiente?	<code>cat /root/PENDIENTES.md head -30</code>	Tu backlog ordenado por prioridad
El stack no responde	<code>/pipeline</code>	Diagnóstico en lenguaje simple + guía
Quiero cambiar código	Pídelo directamente en el chat	Claude lo hace — no necesitas comandos
¿Funcionó el cambio?	<code>make check</code>	Build + health + lint en 1 comando
Quiero hacer un commit	Pídele a Claude que lo haga	Usa el patrón HTTPS correcto automáticamente
Sesión larga, quiero parar	Pide “haz clock-out”	Claude actualiza PENDIENTES, commit, handoff
Nueva sesión, ¿dónde estaba?	<code>cat /root/SESSION_HANDOFF.md</code>	El paso siguiente es el texto literalmente
Quiero que Claude recuerde	“recuerda que...” en el chat	Lo guarda permanentemente en memoria
El harness está desactualizado	<code>/evolve</code>	Audita y actualiza CLAUDE.md automáticamente

Flujo de Sesión

Una sesión tiene 5 fases. Siempre en este orden:

Fase 1: Clock-In

Verificar estado real del stack y qué hay pendiente:

```
make doctor # diagnóstico completo
cat /root/PENDIENTES.md # ver tareas activas
cat /root/SESSION_HANDOFF.md # retomar desde donde terminó
```

Fase 2: Elegir Tarea (UNA sola)

- Prioridad alta primero (items hs-001, hs-002)
- Si surge idea secundaria → registrar en PENDIENTES.md pero **no empezarla**
- Una tarea a la vez previene el “70 % en 3 tareas”

Fase 3: Ejecutar (en el chat)

- Describir qué quieres → Claude investiga + implementa
- Si es algo riesgoso → Claude te pide confirmación
- Si usas voz → dicta directo, Claude normaliza errores tipográficos

Fase 4: Verificar (Definition of Done — 4 criterios)

Una tarea está done cuando se cumplen TODOS estos:

1. ✓ Código implementado
2. ✓ `make check` → sin errores
3. ✓ Evidencia en PENDIENTES.md (commit hash o output real)
4. ✓ `docker compose ps` → todos los servicios Up

Fase 5: Clock-Out

Al terminar sesión:

```
# Pedir: "haz clock-out"
# Claude hará:
# 1. Actualizar PENDIENTES.md con evidencia
# 2. Ejecutar make check para verificar stack sano
# 3. Hacer commit de todo el trabajo
# 4. Crear SESSION_HANDOFF.md
```

1. Slash Commands

Comandos especializados que activan comportamientos automáticos:

Comando	Cuándo usarlo	Qué hace
/evolve	Al final de sesiones largas o si algo no está documentado	Audita los 5 subsistemas del harness, extrae aprendizajes, actualiza CLAUDE.md
/pipeline	El stack falla y no sabes por dónde empezar	Ejecuta <code>make doctor</code> y explica en lenguaje simple qué está mal

Tip para ADHD: Estos comandos son de mantenimiento. El chat normal cubre el 95 % de lo que necesitas.

2. Skills Ruflo — Cuándo Activarlos

Escribe la frase clave en el chat y Claude usa el skill automáticamente:

Diagnóstico y Estado

- “estado del stack” / “qué está corriendo” → `ruflo-status`
- “diagnóstico completo” → `ruflo-doctor`
- “audita la seguridad” → `security-audit`

Memoria y Contexto

- “busca en tu memoria...” / “recuerdas cuando...” → `recall`
- “guarda esto en memoria” / “recuerda que...” → `ruflo-memory`

Código y Documentación

- “genera tests para [archivo]” → `testgen`
- “documenta [módulo]” → `ruflo-docs`
- “revisa este PR” / “revisa el diff” → `code-review`

Automatización y Background

- “programa X cada [intervalo]” → `ruflo-loop`
- “activa autopilot para...” → `autopilot`

Rastreo de Costos

- “cuánto costó esta sesión” → `ruflo-cost`
- “optimiza mi uso de tokens” → `cost-optimize`

3. Árbol de Decisión

¿Qué necesitas ahora?

- **Stack no responde / algo está caído**
 - `/pipeline` → si no basta: `make logs` → `docker compose ps`
- **Quiero cambiar código / funcionalidad**
 - Describir en el chat (Claude implementa) → `make check` → pedir commit
- **¿Qué hay pendiente?**
 - `cat /root/PENDIENTES.md | head -30`
 - Más detalle: `cat /root/PENDIENTES.json`
- **Quiero que Claude recuerde algo**
 - “recuerda que [X]” en el chat → guardado en `~/.claude/projects/.../memory/`
- **Error misterioso**

- `make logs` → `docker compose ps` → pegar error en chat
- **Cuánto estoy gastando**
 - “cuánto costó esta sesión” en el chat
- **Terminar sesión bien**
 - “haz clock-out” en el chat
- **Harness desactualizado**
 - `/evolve` (mejor al final de sesión larga)

4. Tips ADHD+AACC

1. Voz → Acción Sin Fricción

En el chat, usa el prefijo `!` para ejecutar comandos directamente:

```
! make doctor
! cat /root/PENDIENTES.md
! docker compose ps
```

Dicta el comando y el output aparece en la conversación.

2. Una Tarea Activa — El Sistema Te Protege

Cuando surge idea secundaria:

1. Di: “anota esto para después: [idea]”
2. Claude la registra en `PENDIENTES.md`
3. Tú continúas con lo que estabas haciendo

`PENDIENTES.md` es tu buffer externo de memoria.

3. Recuperar Contexto Entre Sesiones en 30 Segundos

```
cat /root/SESSION_HANDOFF.md
```

El archivo contiene literalmente: “el siguiente paso es: [instrucción concreta]”. No necesitas reconstruir nada.

4. Señales de que Claude Entiende vs. Adivina

Claude entiende	Claude adivina
Cita archivos específicos con rutas	Habla en genérico (“deberías...”)
Ejecuta comandos y muestra output real	Dice “probablemente funciona”
Propone verificación concreta	No propone forma de verificar
Pide confirmación antes de cambios	Actúa sin preguntar en cambios riesgosos

Si ves señales de “adivina” → pide: “¿leíste el archivo? muéstrame qué encontraste exactamente”.

5. Opus vs. Sonnet — Cuándo Importa

Situación	Modelo	Por qué
Trabajo diario, cambios de código	Sonnet (default)	Suficiente, mucho más barato
Orquestación multi-agente compleja	Opus 4.7	Mejor descomposición de tareas grandes
Documentos importantes, arquitectura	Opus 4.7	Razonamiento notablemente superior

Para cambiar: escribe `/model` en el chat.

6. Cuando Claude Se Atasca

Si después de 2 intentos el resultado no está bien:

1. Escribe: “Para. Dime exactamente qué estás intentando hacer y qué encontraste.”
2. Claude reporta su estado → identificas si es comprensión o ejecución
3. Si sigue atascado (3+ intentos) → escala a escritura directa o sub-agente

7. El Chat Es Tu Terminal Principal

No abras terminales separadas. En el chat puedes:

- Pedir que ejecute comandos
- Ver outputs directamente
- Pedir que interprete resultados
- Pedir que tome acción basada en lo que vio

5. Servicios y Puertos — Referencia Rápida

Servicio	URL Pública	Puerto	Estado Esperado
Hermes Agent	hermes.el80.space	8080	Up + <code>/health 200</code>
LiteLLM Router	litellm.el80.space	4000	Up + <code>/health 200</code>
Website	docs.el80.space	3001	Up
Grafana	grafana.el80.space	3000	Up
FileBrowser	files.el80.space	8095	Up
Whisper STT	— (interno)	9000	Up
Prometheus	— (interno)	9090	Up

Verificar todo:

```
make health-check # ver estado rápido
make status # docker compose ps
```

6. Comandos Make — Cheat Sheet

```
make doctor # Diagnóstico completo (6 pasos)
make check # Verificación rápida: build + health + lint
make health-check # Solo health
make build-check # Solo build Next.js
make lint-check # Solo lint Python
```

```
make status # docker compose ps
make logs # Últimas 40 líneas de cada servicio
make clean-tmp # Limpiar /tmp/claude_progress y /tmp/*.md
```

7. Glosario Mínimo

- **harness:** Conjunto de archivos que hacen que Claude trabaje bien: CLAUDE.md, AGENTS.md, PENDIENTES.json, Makefile, hooks
- **skill:** Capacidad especializada que se activa con frases clave (“genera tests”). No necesitas saber el nombre técnico
- **swarm:** Varios agentes trabajando en paralelo en subtareas. Claude lo orquesta automáticamente
- **sub-agente:** Agente que Claude lanza internamente. Tú no los controlas directamente
- **PENDIENTES.json:** Backlog estructurado con id, prioridad, estado, verificación, evidencia. Claude lo actualiza
- **SESSION_HANDOFF.md:** Nota de Claude a Claude: “lo que quedó pendiente y el siguiente paso concreto”. Recuperación en 30 segundos
- **Definition of Done:** 4 criterios para considerar una tarea terminada (implementación + verificación + evidencia + stack Up)
- **clock-in / clock-out:** Checklist de inicio y cierre de sesión. Garantiza estado conocido
- **MCP:** Protocolo que permite a Claude usar herramientas externas (Google Drive, Gmail, browser, DB). Ya configurado

Última actualización: 25 de mayo de 2026 · Hermes Stack · erikjosehrndz-crypto